Project Overview

Over de komende middagen bouw je je eigen mini-CDI container in Java. Je begint klein, met een registry waarin je objecten bewaart, en bouwt daar stap voor stap een container omheen die zelf uitzoekt welke objecten er nodig zijn en hoe ze aan elkaar gekoppeld worden. Aan het eind van de rit kun je je container vergelijken met wat CDI-implementaties als Weld of Quarkus ArC doen.

Je gebruikt alleen de standaard Java bibliotheek, Maven en JUnit. Geen Spring, geen Weld, geen Guice. Want zodra je met een ander framework werkt, zie je de trucjes niet meer. (De enige uitzondering is de bonus van middag 4, waarin je in een aparte module vergelijkt met Weld.)

De zeven middagen in vogelvlucht:

Middag	Thema	Wat je container erbij krijgt
1	Fundering	Registry, `@Inject`, constructor injection, duidelijke fouten
2	Beans vinden	Classpath scanning, interfaces, plugin beans uit een externe jar
3	Functional Java	Refactor met streams, `Provider<T>`, `Optional`, records en sealed types
4	Lifecycle en scopes	`@PostConstruct`, `@PreDestroy`, request scope, client proxy, `@Logged` interceptor
5	Concurrency	Thread-safe `getBean`, request scope per thread, stateful beans
6	Performance	Eigen JFR events, profilen, geheugenlekken opsporen
7	Afronding	Integratie, presentatie en quiz

Afspraken voor het project

- **Java-versie:** JDK 21 of nieuwer. Middag 3 gebruikt pattern matching in switch, en dat is pas vanaf Java 21 definitief.

- **API:** deze methodes heten in elke container hetzelfde, zodat tests, het startpakket van middag 6 en de modeloplossingen bij elkaar passen:
  - `<T> void register(Class<T> type, T instance)` (middag 1)
  - `<T> T getBean(Class<T> type)` (middag 1)
  - `void registerClass(Class<?> type)` (middag 1)
  - `<T> void bind(Class<T> type, Class<? extends T> implementation)` (bonus middag 1)
  - `void scan(String packageName)` en `void loadPlugin(Path jarFile)` (middag 2)
  - `beginRequest()`, `endRequest()` en `close()` (middag 4)
- **Fouten:** elke fout van je container is een `ContainerException` of een subklasse daarvan, zoals `UnsatisfiedDependencyException` en `CircularDependencyException`. `ContainerException` extends `RuntimeException`. Dat is nodig voor de proxies van middag 4: een checked exception uit een proxy komt bij de aanroeper aan als `UndeclaredThrowableException`.
- **Lazy:** registreren en scannen controleren niets. De container kijkt pas bij `getBean` wat een klasse nodig heeft, en pas dan komen fouten naar boven.
- **Alleen geregistreerd:** de container maakt alleen klassen aan die geregistreerd zijn (met `registerClass`, `bind`, scan of `loadPlugin`). Een dependency die niet geregistreerd is, is een ontbrekende dependency, ook als de klasse bestaat en een bruikbare constructor heeft.
- **Testklassen:** voorbeeldklassen die bij het startproject horen staan in `src/main/java` onder `nl.topicus.cdi.example`. Klassen die je alleen voor een test maakt, zet je in `src/test/java` onder `nl.topicus.cdi.testbeans`, met een eigen subpackage per scenario. Zo scant een test precies wat hij nodig heeft, en komen testklassen niet per ongeluk mee in een scan van `example`.
- **Tests blijven groen:** tests uit eerdere middagen blijven groen, tenzij de opdracht expliciet zegt dat een gedrag verandert. Een test die bewust een probleem laat zien, geef je een naam die met `documenteert` begint. Zo'n test zet je om zodra je het probleem oplost, of je laat hem draaien op een aparte klasse die het probleem houdt.

Afternoon 1: De Fundering — Registry, Annotaties en Constructor Injection

Goal

Oh nee, de applicatie start niet op en de logging zegt alleen: `Unsatisfied dependency for type PaymentGateway`. Wie snapt wat de container doet bij het opstarten, vindt zo'n fout in een paar minuten. Wie het niet snapt, begint met willekeurig annotaties toevoegen en hopen dat het werkt.

Vanmiddag bouw je de kern van een DI container. Je maakt een registry, een eigen `@Inject` en `@ApplicationScoped` annotatie, en een `Container` die via reflection constructors aanroept met de juiste dependencies. Daarna zorg je dat de container bruikbare foutmeldingen geeft bij ontbrekende en circulaire dependencies.

Learning goals

- Eigen annotaties definiëren en tijdens runtime uitlezen met reflection
- Een container implementeren die objecten aanmaakt via constructor injection
- Foutscenario's debuggen en omzetten in duidelijke foutmeldingen voor de gebruiker van je container
- Beoordelen waarom circulaire dependencies bij constructor injection niet op te lossen zijn en welke ontwerpkeuzes je daarvoor hebt

Project afternoon parts

Het project van deze middag bevat **5 delen**.

Je krijgt een startproject met een lege `Container` klasse, een leeg annotatiepackage, een aantal voorbeeldklassen die je framework gaan gebruiken en een paar tests. De voorbeeldklassen staan onder `nl.topicus.cdi.example`:

- `order`: een keten die goed gaat. `OrderService` heeft een `PaymentService` en een `InvoiceService` nodig, en die hebben allebei een `PaymentRepository` nodig

- `broken`: `BrokenService` heeft een `ExternalClient` nodig, een klasse die bestaat maar nergens geregistreerd wordt. `FailingService` gooit een exception in zijn constructor
- `cycle`: `CycleA` heeft een `CycleB` nodig en `CycleB` een `CycleA`

Part 1: Registry bouwen

Bouw een `Container` klasse waarin je handmatig objecten kunt registreren en ophalen op basis van hun type, met de methodes `register(Class<T> type, T instance)` en `getBean(Class<T> type)`. Dit is de basis waarop de rest van de middag voortbouwt. Denk na over wat de container moet doen als iemand een type opvraagt dat er niet is, en hoe de aanroeper een `PaymentService` terugkrijgt zonder zelf te casten.
Maak ook meteen je `ContainerException`. Alle fouten die je container vanaf nu gooit, zijn deze exception of een subklasse ervan.

Success criteria

- Een geregistreerd object kan worden opgehaald op zijn type
- De aanroeper hoeft het resultaat niet zelf te casten
- Twee keer hetzelfde type opvragen levert hetzelfde object op
- Een onbekend type opvragen geeft een `ContainerException`, geen `null`
- `ContainerException` is een `RuntimeException`
- Een unit test dekt bovenstaande gevallen

Part 2: Annotaties definiëren

Maak de annotaties `@Inject` en `@ApplicationScoped`. Met `@Inject` markeer je welke constructor de container moet gebruiken. Met `@ApplicationScoped` markeer je dat een klasse door de container beheerd wordt. De annotaties moeten tijdens runtime zichtbaar zijn en op de juiste plek gebruikt kunnen worden.

Pas de annotaties toe op de voorbeeldklassen in `order` en `cycle`, en op `BrokenService` en `FailingService`. `ExternalClient` krijgt bewust geen annotatie.

Success criteria

- Beide annotaties bestaan en zijn geschikt voor hun doel (constructors respectievelijk klassen)

- `@Inject` op een veld of methode geeft een compile error (dit controleer je met de hand, een unit test kan geen compile error testen)
- Een test leest met reflection uit dat `OrderService` een `@Inject` constructor heeft en `@ApplicationScoped` is

Part 3: Constructor injection via reflection

Laat de container zelf een object aanmaken. Voeg `registerClass(Class<?> type)` toe: je registreert nu een klasse in plaats van een kant-en-klaar object. Als iemand die klasse opvraagt, zoekt de container uit welke constructor gebruikt moet worden, welke dependencies daarvoor nodig zijn, en maakt die eerst aan als ze geregistreerd zijn.
Twee afspraken waar latere middagen op bouwen:

- `registerClass` controleert niets. Pas bij `getBean` kijkt de container wat een klasse nodig heeft.
- De container maakt alleen geregistreerde klassen aan. Is een dependency niet geregistreerd, dan is dat een ontbrekende dependency (Part 4), ook als de klasse een prima constructor heeft.

Welke constructor de container gebruikt: de constructor met `@Inject`. Heeft een klasse geen `@Inject` constructor, dan alleen een parameterloze constructor. Dat is ook de regel die CDI hanteert.

Een `OrderService` die een `PaymentService` nodig heeft, die op zijn beurt een `PaymentRepository` nodig heeft, moet met één aanroep volledig opgebouwd worden.

Success criteria

- Na `registerClass` voor alle klassen in order geeft `container.getBean(OrderService.class)` een volledig geïnjecteerde `OrderService` terug
- Alle klassen in de keten worden aangemaakt zonder dat je ze met de hand doorgeeft
- Dezelfde klasse twee keer opvragen levert dezelfde instance op
- `PaymentRepository`, die door twee klassen gebruikt wordt, bestaat maar één keer
- Een klasse met alleen een parameterloze constructor werkt zonder `@Inject`
- Een klasse met twee `@Inject` constructors, of zonder `@Inject` en zonder parameterloze constructor, geeft een `ContainerException` die uitlegt wat er mis is
- `registerClass(BrokenService.class)` gooit geen exception
- Een test dekt de hele keten

Part 4: Ontbrekende dependencies

Zorg dat de container een bruikbare foutmelding geeft als een dependency niet aangemaakt kan worden. Een goede foutmelding vertelt niet alleen wat er ontbreekt, maar ook welke klasse erom vroeg.

Zoek ook uit hoe je omgaat met fouten die tijdens het aanmaken optreden, zoals bij `FailingService`, waarvan de constructor zelf een exception gooit.

Success criteria

- `getBean(BrokenService.class)` geeft een `UnsatisfiedDependencyException` (een subklasse van `ContainerException`), geen `NullPointerException` of kale reflection exception
- De melding bevat zowel het ontbrekende type (`ExternalClient`) als de klasse die het nodig had (`BrokenService`)
- `getBean(FailingService.class)` geeft een `ContainerException` met de originele exception als cause
- Er is geen `InvocationTargetException` in de melding die de gebruiker te zien krijgt
- Een test bewijst elk van deze gevallen

Part 5: Circulaire dependencies

Als `CycleA` een `CycleB` nodig heeft en `CycleB` een `CycleA`, kan de container geen van beide aanmaken. Zonder extra maatregelen krijg je een `StackOverflowError`, en dat is een van de vervelendste manieren om een fout te ontdekken. Zorg dat je container de cirkel herkent voordat het misgaat en de gebruiker vertelt wat er aan de hand is.

Voor de langere cirkel maak je zelf drie klassen in `nl.topicus.cdi.testbeans`.

Success criteria

- Een directe cirkel (`CycleA` → `CycleB` → `CycleA`) geeft een `CircularDependencyException` (een subklasse van `ContainerException`) en geen `StackOverflowError`
- Een langere cirkel (`A` → `B` → `C` → `A`) wordt ook herkend
- De melding toont het hele pad, bijvoorbeeld `A → B → C → A`

- De keten in `order`, die `PaymentRepository` op twee plekken gebruikt, wordt niet foutief als cirkel gezien
- Na een mislukte poging blijft de container bruikbaar voor andere klassen
- Een test dekt de directe en de langere cirkel

Bonus Challenge (Optional)

Laat je container ook werken met interfaces. Tot nu toe vraag je steeds een concrete klasse op, terwijl echte applicaties vaak om een PaymentGateway vragen en de container moeten laten bepalen welke implementatie dat wordt.

- Voeg `bind(Class<T> type, Class<? extends T> implementation)` toe en laat `bind(PaymentGateway.class, MockGateway.class)` gevolgd door `getBean(PaymentGateway.class)` een `MockGateway` teruggeven
- Laat `@Inject` constructors met een interface-parameter werken
- Bedenk wat de container moet doen als er twee implementaties zijn, en implementeer een duidelijke foutmelding voor dat geval

Waarom heet deze methode niet gewoon ook `register`? Probeer `register(PaymentGateway.class, MockGateway.class)` eens met de signatuur uit Part 1 en verklaar wat de compiler zegt. Wat zou er gebeuren als `register` als tweede parameter een `Object` had geaccepteerd?